

Servomotores da Série M17 - FICHA TÉCNICA



Controlo de Movimento Tudo-em-Um, Simples e Acessível
Da Educação à Inovação



Introdução

Os Servomotores da Série M17 são soluções de controlo de movimento tudo-em-um que integram um motor, um driver de motor, um controlador de movimento e um encoder num único conjunto compacto. Estes servomotores dispõem de uma interface de comunicação RS-485, permitindo ligar várias unidades em cadeia (daisy-chain) e controlá-las a partir de um único ponto de ligação. Disponível em quatro modelos - M17-60, M17-48, M17-40 e M17-34 - a série oferece opções flexíveis para corresponder a requisitos específicos de binário (torque) e de tamanho, mantendo características de controlo consistentes. Cada Servomotor da Série M17 inclui funcionalidades de controlo sofisticadas, incluindo múltiplos modos de operação, capacidades de autocalibração e monitorização de estado integrada através de LEDs indicadores. Os motores podem ser facilmente controlados a partir de qualquer plataforma, incluindo Mac, PC, Raspberry Pi ou Arduino (necessitando apenas de um adaptador RS485 de baixo custo), o que os torna ideais tanto para aplicações educativas como industriais. A série suporta controlo de posição preciso com perfis de movimento trapezoidais, modo de controlo em malha fechada e tratamento abrangente de erros. Com dimensões de montagem NEMA 17 normalizadas, uma ampla gama de tensão (12-24V) e um protocolo de comunicação robusto, os Servomotores da Série M17 proporcionam uma solução fiável e flexível para aplicações que exigem controlo de movimento preciso, desde a robótica e as máquinas CNC até equipamentos de teste automatizados e instrumentos científicos.

Envie-nos o Seu Feedback

Se encontrar erros neste documento ou tiver questões, informe-nos em: tutorial.gearotons.com/feedback. Agradecemos muito as suas sugestões sobre como podemos melhorar o produto ou a documentação. Muito obrigado!

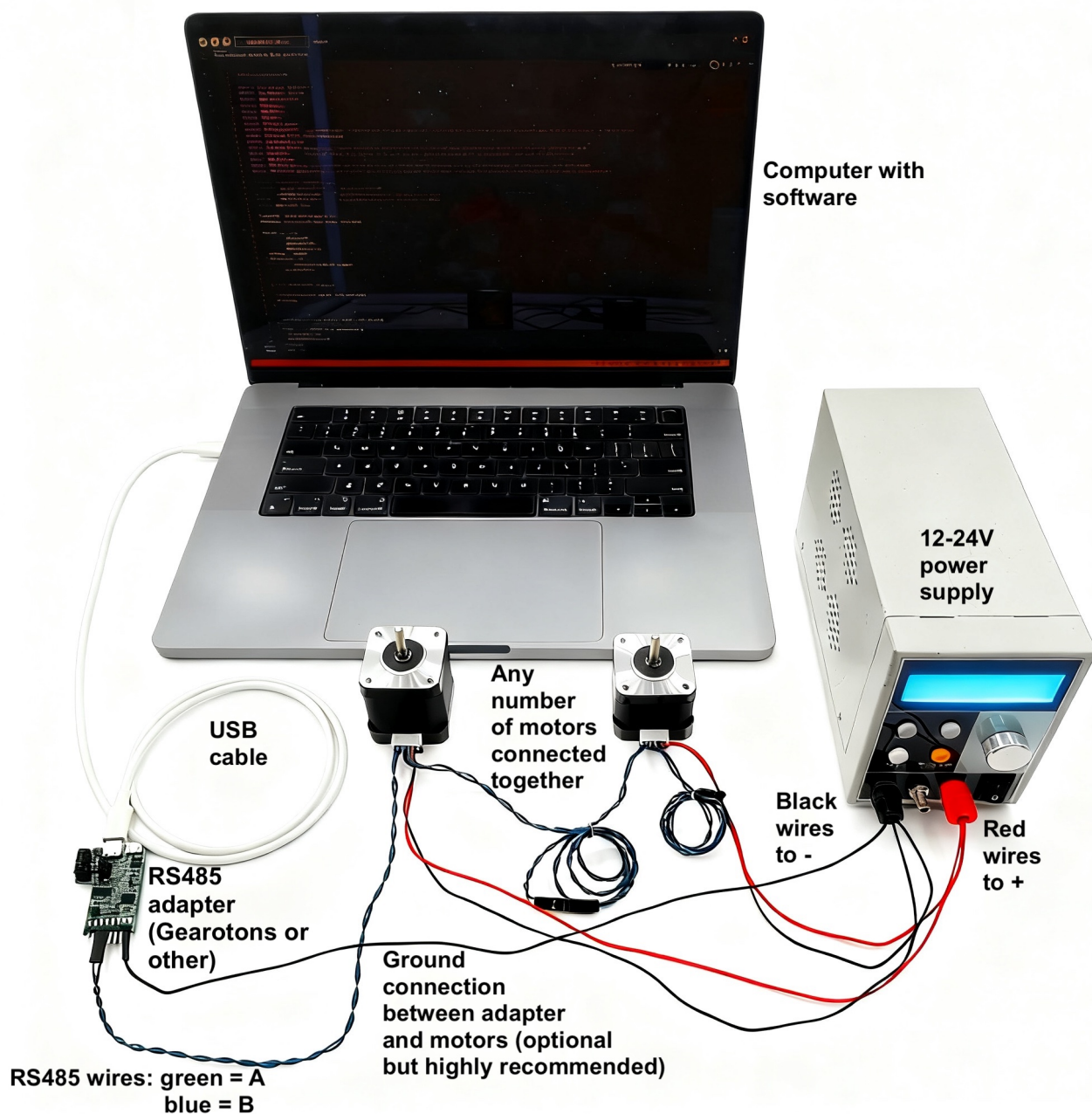


Visite a nossa Página de Feedback

Características Principais

- Elevado nível de integração, combinando um motor, um driver de motor, um sistema de controlo de movimento e um encoder
- Controle qualquer número de motores a partir de um único controlador simples através de uma interface de comunicação RS-485
- Controlo através de comandos de alto nível como "Enable mosfets" (ativar MOSFETs) e "Trapezoid move" (movimento trapezoidal), evitando o controlo DIR/STEP com temporização crítica
- Formato compacto, praticamente do mesmo tamanho de um motor de passo NEMA 17 com as mesmas especificações (sem protuberâncias)
- Dimensões de montagem NEMA 17 normalizadas
- Ampla gama de tensão (12-24V) para opções de alimentação flexíveis
- Controlo em malha fechada de alta precisão, com encoder integrado e malha de controlo PID a funcionar a 32 kHz
- Muito mais eficiente em termos energéticos em controlo de malha fechada do que um motor de passo convencional
- Proteção integrada contra sobrecorrente, sobretensão e sobreaquecimento
- A velocidade máxima pode atingir 560 RPM
- Relação binário/peso igual à de um motor de passo equivalente
- Compatível com uma grande variedade de interfaces e hardware, como Raspberry Pi, Arduino, ESP32, Mac e PC
- Esforçamo-nos por fornecer excelente documentação e tutoriais para o pôr a funcionar rapidamente
- Fornecemos documentação adequada para IA, caso queira que a sua IA favorita faça todo o trabalho por si
- Adequado para robótica, CNC, automação, instrumentos científicos, dispositivos de teste, impressoras 3D e muito mais
- Disponível em diferentes tamanhos, para que possa encontrar o binário e o preço certos para a sua aplicação

Diagrama de Ligações



Sistema de Unidades

Os Servomotores da Série M17 utilizam determinadas unidades internas para poderem efetuar de forma eficiente os cálculos associados ao movimento (usando aritmética de números inteiros). É da responsabilidade do software de controlo suportar múltiplas unidades de medida para as várias grandezas. As nossas bibliotecas Python e Arduino tratam automaticamente das conversões de unidades, permitindo-lhe trabalhar com as unidades da sua preferência. Abaixo encontram-se as unidades suportadas para cada grandeza:

Grandeza	Unidades Disponíveis
Tempo	timesteps, seconds, milliseconds, minutes, microseconds
Posição	shaft rotations, degrees, radians, encoder counts
Velocidade	rotations per second, rpm, degrees per second, radians per second, counts per second, counts per timestep
Aceleração	rotations per second squared, rpm per second, degrees per second squared, radians per second squared, counts per second squared, counts per timestep squared
Corrente	internal current units, milliamps, amps
Tensão	millivolts, volts
Temperatura	celsius, fahrenheit, kelvin

Guia de Iniciação

Para o ajudar a começar a utilizar o seu Servomotor da Série M17, disponibilizamos um guia online completo que cobre tudo, desde a configuração inicial até implementações avançadas do protocolo. Este guia inclui:

- Instruções de configuração passo a passo
- Documentação detalhada do protocolo de comunicação
- Exemplos de programação e excertos de código
- Descrição dos códigos de erro
- Dicas de resolução de problemas e boas práticas



[Clique aqui para o Guia de Iniciação: tutorial.gearotons.com](https://tutorial.gearotons.com)

LEDs Indicadores e Botões

O servomotor tem dois LEDs de estado (verde e vermelho). O LED verde pisca lentamente para mostrar um sinal de vida (heartbeat) e rapidamente para indicar que está a ser executado o bootloader em vez da aplicação. O LED vermelho acende-se brevemente para mostrar comunicação no barramento e indica códigos de erro fatais piscando um determinado número de vezes.

O servomotor tem dois botões identificados como "Reset" (reiniciar) e "Test" (testar). O botão Reset reinicia o microcontrolador interno e todo o estado regressa aos valores predefinidos. O botão Test faz o motor rodar. Prima brevemente para o fazer rodar num sentido e prima durante mais de 0,3 segundos e solte para o fazer rodar no sentido oposto. Mantenha premido durante pelo menos 2 segundos e solte para colocar o motor em modo de malha fechada. Mantenha premido durante mais de 15 segundos e solte para que o motor execute uma calibração de si próprio. Tenha em atenção que o motor roda durante a calibração e tem de poder rodar livremente para que a calibração seja bem-sucedida, pelo que deve remover quaisquer cargas antes de efetuar esta operação.

Visão Geral do Protocolo de Comunicação

Os Servomotores da Série M17 utilizam RS-485 para a comunicação, permitindo ligar vários motores em cadeia (daisy-chain). Cada motor tem um endereço único de 64 bits atribuído de fábrica, para que possa ser endereçado individualmente num barramento RS-485. A cada motor pode também ser atribuído um alias único de 1 byte (0-251), de forma a poupar bytes no pacote de comunicação e ainda assim obter um controlo individual semelhante. Existe também um alias de difusão (broadcast), que é o 255. O protocolo suporta um conjunto abrangente de comandos para controlo de movimento, configuração e monitorização de estado. A atualização de firmware através da interface RS-485 é suportada. A velocidade de comunicação está fixada em 230400 baud.

Resumo de Referência de Comandos

Para a fonte de informação atualizada sobre todos os comandos disponíveis, pode consultar este documento.



https://github.com/tomrodinger/servomotor/blob/main/python_programs/servomotor/motor_commands.json

Também pode executar este comando:

```
pip3 install servomotor # run this just once to install the library and programs
servomotor_command -c
```

Isto imprime a informação contida no ficheiro motor_commands.json de uma forma mais legível e fornece algumas indicações de utilização para enviar comandos ao motor a partir da linha de comandos.

Controlo Básico

Comando	Descrição
Disable MOSFETs	Desativa os MOSFETs (note que os MOSFETs estão desativados por predefinição após a ligação inicial).
Enable MOSFETs	Ativa os MOSFETs.
Reset time	Repõe o tempo absoluto a zero (chame este comando primeiro, antes de emitir quaisquer comandos de movimento).
Emergency stop	Executa uma paragem de emergência (para todo o movimento, desativa os MOSFETs e limpa a fila).
Zero position	Define a posição atual como a posição zero (a origem).
System reset	Reinicia o sistema e entra no bootloader. O motor reinicia imediatamente e entra no bootloader. Se nenhum comando for enviado dentro de um curto período de tempo, o motor sai do bootloader e executa a aplicação desde o início.

Controlo de Movimento

Comando	Descrição
Trapezoid move	Move imediatamente para a posição indicada utilizando a velocidade atualmente definida (a velocidade é definida por um comando separado).
Go to position	Move para a posição indicada durante o intervalo de tempo especificado. São aplicadas aceleração e desaceleração para tornar o movimento suave.
Homing	Efetua o retorno à origem (homing) do motor (por outras palavras, move até colidir e depois para imediatamente).
Go to closed loop	Entra no modo de controlo de posição em malha fechada.
Move with acceleration	Roda o motor com a aceleração especificada.
Move with velocity	Roda o motor com a velocidade especificada.
Multimove	Compõe múltiplos movimentos a serem executados um após o outro. Note que, se a fila ficar vazia depois de todos os movimentos serem executados e o motor não estiver parado, é desencadeado um erro fatal.

Configuração

Comando	Descrição
Set maximum velocity	Define a velocidade máxima (não utilizada de momento).
Set maximum acceleration	Define a aceleração máxima.
Start calibration	Inicia uma calibração, que determina os valores médios dos sensores de Hall e se estes estão a funcionar corretamente.
Set maximum motor current	Define a corrente máxima do motor e a corrente máxima de regeneração. Os valores têm efeito imediato, mas não são guardados em memória não volátil, pelo que voltam à predefinição do firmware após um reset.
Set safety limits	Define os limites de segurança (para impedir que o movimento exceda os limites estabelecidos).
Test mode	Define ou aciona um determinado modo de teste. De momento, esta funcionalidade está pouco documentada. Não a utilize, a menos que seja um programador a trabalhar em casos de teste. Modos notáveis: 0 = limpa todos os modos de teste; 1-9 = modos de teste do motor; 10-13 = modos de teste dos LEDs (bloqueiam o dispositivo); 14-73 = acionam os erros fatais 0-59; 74 = define o limiar de proteção contra sobretensão para 22 V (teste de produção; deve disparar com uma alimentação de 24 V); 75 = define o limiar de proteção contra sobretensão para 26 V (teste de produção; não deve disparar com uma alimentação de 24 V); 76 = eleva o limite de corte por sobreaquecimento para cerca de 90 graus C (teste de margem térmica em produção; o limite de corte predefinido é de cerca de 80 graus C). Um reset do sistema restaura os limiares predefinidos de sobretensão e de sobreaquecimento.
Set PID constants	Define as constantes PID da malha de controlo que tenta manter a trajetória de movimento.
Set max allowable position deviation	Define o número de micropassos que a posição real do motor (medida pelos sensores de Hall) pode desviar-se da posição desejada. É gerado um erro fatal se este valor for excedido.
CRC32 control	Ativa ou desativa a verificação CRC32 dos comandos.

Estado e Monitorização

Comando	Descrição
Get current time	Obtém o tempo absoluto atual.
Get n queued items	Obtém o número de itens atualmente na fila de movimentos (se este número ficar demasiado grande, não coloque mais comandos de movimento na fila).
Get hall sensor position	Obtém a posição medida pelos sensores de Hall (esta deverá ser a posição real do motor; se tudo estiver bem, será aproximadamente igual à posição desejada).
Get status	Obtém o estado do motor.
Control hall sensor statistics	Liga ou desliga a recolha de estatísticas dos sensores de Hall e repõe as estatísticas.
Get hall sensor statistics	Lê as estatísticas recolhidas dos sensores de Hall. Isto é útil para verificar a integridade dos sensores de Hall e o ruído no sistema.
Get position	Obtém a posição desejada atual (que pode diferir ligeiramente da posição real medida pelos sensores de Hall).
Get comprehensive position	Obtém a posição desejada do motor, a posição dos sensores de Hall e a posição do encoder externo, tudo numa única leitura.
Get supply voltage	Obtém a tensão medida da fonte de alimentação.
Get max PID error	Obtém os valores mínimo e máximo do erro observados na malha de controlo PID desde a última leitura.
Get temperature	Obtém a temperatura medida do motor.
Get debug values	Obtém valores de depuração, incluindo parâmetros de controlo do motor, tempos do profiler, dados dos sensores de Hall e outras informações de diagnóstico.
Get communication statistics	Obtém e, opcionalmente, repõe o contador de erros CRC32.

Gestão de Dispositivos

Comando	Descrição
Time sync	Envia o tempo do mestre para o motor, para que este possa sincronizar o seu próprio relógio (faça isto 10 vezes por segundo).
Get product specs	Obtém a frequência de atualização (o inverso do passo de tempo).
Detect devices	Deteta todos os dispositivos ligados à interface RS485. Os dispositivos identificam-se num instante aleatório dentro de um segundo. Uma colisão é possível, mas improvável. Pode repetir esta operação se suspeitar de uma colisão (por exemplo, se tiver dispositivos ligados que não foram descobertos no espaço de um a dois segundos).
Set device alias	Define o alias do dispositivo.
Get product info	Obtém as informações do produto.
Firmware upgrade	Atualiza a memória flash do servomotor. Antes de emitir um comando de atualização de firmware, é necessário efetuar alguns cálculos, como mostrado nos exemplos.
Get product description	Obtém a descrição do produto.
Get firmware version	Obtém a versão do firmware ou a versão do bootloader, consoante o modo em que o dispositivo se encontra. Este comando também devolve os bits de estado, em que o bit menos significativo indica se o dispositivo está atualmente no bootloader (=1) ou no firmware principal (=0).
Ping	Envia um payload com quaisquer dados e o dispositivo responde com os mesmos dados.
Vibrate	Faz com que o motor varie a tensão rapidamente e, por conseguinte, vibre (ou deixe de vibrar).
Identify	Identifique o seu motor enviando este comando. O LED verde do motor pisca rapidamente durante 3 segundos.

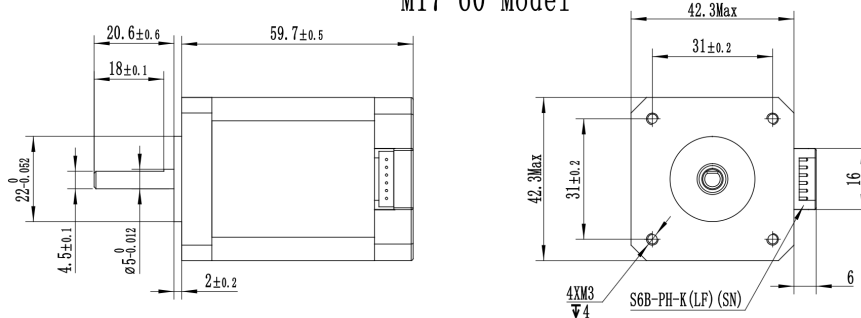
Outros

Comando	Descrição
Capture hall sensor data	Inicia o envio de dados dos sensores de Hall (trabalho em curso; não envie este comando).
Read multipurpose buffer	Lê o conteúdo do buffer multiusos (o buffer é utilizado para dados gerados durante a calibração, ao entrar no modo de malha fechada e ao capturar dados dos sensores de Hall).

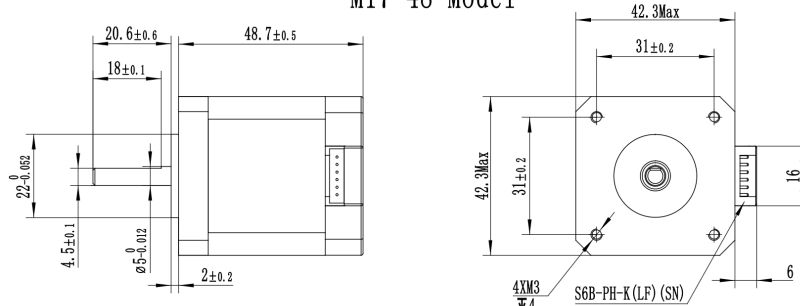
Especificações Mecânicas

Parâmetro	M17-60	M17-48	M17-40	M17-34
Dimensões (CxL)	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm
Altura	59.7 mm	48.7 mm	40.1 mm	33.5 mm
Comprimento do Veio	20.6 mm	20.6 mm	20.6 mm	20.6 mm
Peso	470g	360g	285g	210g
Classe de Proteção	IP20	IP20	IP20	IP20

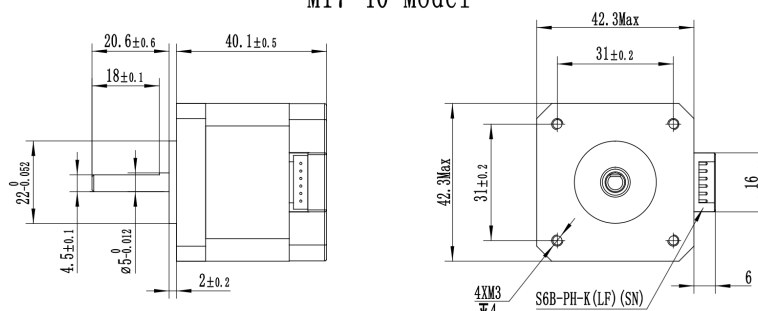
M17-60 Model



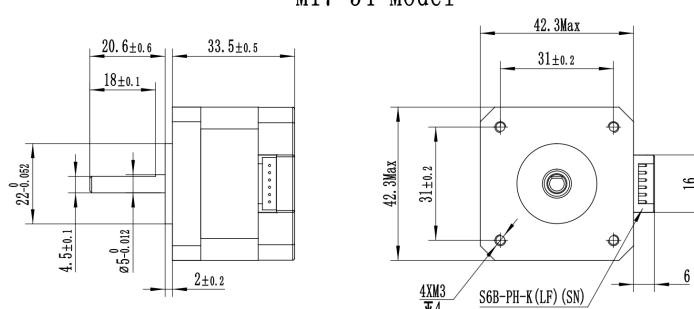
M17-48 Model



M17-40 Model



M17-34 Model



Especificações Técnicas

Parâmetro	M17-60	M17-48	M17-40	M17-34
Tensão de Funcionamento	12-24V	12-24V	12-24V	12-24V
Binário Nominal	0.65 N.m	0.55 N.m	0.42 N.m	0.28 N.m
Velocidade Máxima	560 RPM	560 RPM	560 RPM	560 RPM
Corrente Máxima	1.1A	1.1A	1.1A	1.0A
Potência Nominal	26.4W	26.4W	26.4W	24.0W

Condições de Funcionamento

Parâmetro	Especificação
Temperatura de Funcionamento	0C a +80C
Temperatura de Armazenamento	-20C a +60C
Gama de Humidade	20% a 80% RH (sem condensação)
Ambiente de Instalação	Apenas para utilização em interiores

Biblioteca Python

A nossa biblioteca Python fornece uma interface completa para controlar os Servomotores da Série M17. A biblioteca trata de todos os protocolos de comunicação de baixo nível e das conversões de unidades, facilitando a integração do controlo de servomotores nos seus projetos Python.

Instalação

Instale a biblioteca servomotor utilizando o pip:

```
pip3 install servomotor
```

Exemplo: Controlo Básico do Motor

O exemplo seguinte demonstra como ligar a um servomotor, ativar os MOSFETs e executar um movimento trapezoidal:

```
#!/usr/bin/env python3
"""
Minimal trapezoid move: rotate 1 turn in 1 second.
Edit ALIAS below if needed. Uses rotations and seconds.
"""
import time, servomotor
from servomotor import communication

# Hard-coded settings for a minimal demo
ALIAS = 'X' # Device alias, change if needed
SERIAL_PORT = "/dev/tty.usbserial-110" # Serial device path; change if needed (e.g., "COM3" on
# Windows)
DISPLACEMENT_ROTATIONS = 1.0 # 1 rotation
DURATION_SECONDS = 1.0 # 1 second
DELAY_MARGIN = 0.10 # +10% wait margin because the motor's clock is not
# perfectly accurate

communication.serial_port = SERIAL_PORT # if you comment this out then the program
# should prompt you for the serial port or it will use
# the last used port from a file

servomotor.open_serial_port()

m = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations", verbose=0)
m.enable_mosfets()
m.trapezoid_move(DISPLACEMENT_ROTATIONS, DURATION_SECONDS)
time.sleep(DURATION_SECONDS * (1.0 + DELAY_MARGIN))
m.disable_mosfets()

servomotor.close_serial_port()
```

Características Principais

- Suporte para todos os comandos do motor
- Conversão automática de unidades para tempo, posição, velocidade, aceleração, corrente, tensão e temperatura
- Tratamento de erros e monitorização de estado
- Suporte para vários motores no mesmo barramento
- Comece a trabalhar com apenas algumas linhas de código

Documentação

Para a documentação completa da API e mais exemplos, visite:
https://github.com/tomrodinger/servomotor/tree/main/python_programs

Biblioteca Arduino

A biblioteca Arduino para os Servomotores da Série M17 fornece uma interface fácil de utilizar para controlar motores a partir de placas Arduino. A biblioteca suporta comunicação série por hardware e inclui todas as funções essenciais de controlo do motor.

Instalação

Instale a biblioteca através do Gestor de Bibliotecas do Arduino:

- 1. Abra o Arduino IDE
- 2. Vá a Sketch → Incluir Biblioteca → Gerir Bibliotecas
- 3. Pesquise por "Servomotor"
- 4. Encontre a biblioteca chamada "Servomotor by Gearotons" e clique em Instalar



Instalação Manual

Em alternativa, se for um utilizador avançado, pode obter o código da biblioteca Arduino aqui: https://github.com/tomrodinger/Servomotor_Arduino_Library

Exemplo: Controlo Básico do Motor

Este exemplo mostra como inicializar um motor, ativar os MOSFETs e executar um movimento trapezoidal:

```

// Minimal Arduino example: Trapezoid move using built-in unit conversions
// Goal: spin the motor exactly 1 rotation in 1 second, then stop.
// Sequence:
// enable MOSFETs -> trapezoidMove(1.0 rotations, 1.0 seconds) -> wait 1.1s -> disable MOSFETs.
//
// Notes:
// - This uses the library's unit conversion (no raw counts/timesteps).
// - Configure Serial1 pins for your board (ESP32 example pins below).
// - Motor is created AFTER Serial1.begin(...) so hardware UART pins are set first.

#include <Servomotor.h>

#define ALIAS 'X' // Device alias
#define BAUD 230400 // RS485 UART baud rate
#define DISPLACEMENT_ROTATIONS 1.0f // 1 rotation
#define DURATION_SECONDS 1.0f // 1 second
#define TOLERANCE_PERCENT 10 // +10% wait margin because the motor's clock is not
// perfectly accurate
#define WAIT_MS ((unsigned long)(DURATION_SECONDS * 1000.0f * (100 + TOLERANCE_PERCENT) / 100))

// Example RS485 pins for ESP32 DevKit (change as needed for your board)
#if defined(ESP32)
#define RS485_TXD 4 // TX pin to RS485 transceiver
#define RS485_RXD 5 // RX pin from RS485 transceiver
#endif

void setup() {
  Serial.begin(115200); // Console serial for debugging

  // Create the motor; serial port opens on first instantiation.
#if defined(ESP32)
  Servomotor motor(ALIAS, Serial1, RS485_RXD, RS485_TXD);
#else
  Servomotor motor(ALIAS, Serial1);
#endif

  // Use units: rotations for position, seconds for time
  motor.setPositionUnit(PositionUnit::SHAFT_ROTATIONS);
  motor.setTimeUnit(TimeUnit::SECONDS);

  motor.enableMosfets();
  motor.trapezoidMove(DISPLACEMENT_ROTATIONS, DURATION_SECONDS);
  delay(WAIT_MS);
  motor.disableMosfets();
}

void loop() {
}

```

A Nossa Missão

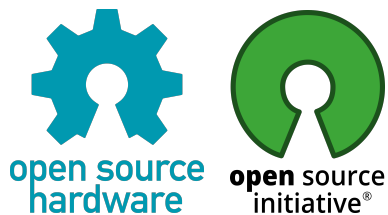
Na Gearotons, acreditamos que o futuro pertence a quem compreende e consegue trabalhar lado a lado com a inteligência artificial e os sistemas de automação. A nossa missão é capacitar a próxima geração de inovadores, tornando a tecnologia avançada de controlo de movimento acessível, compreensível e económica para criadores, educadores e engenheiros em todo o mundo. Estamos empenhados em derrubar as barreiras que tradicionalmente separaram os estudantes e os entusiastas das ferramentas de automação de nível profissional. Ao fornecer soluções integradas que eliminam a complexidade sem sacrificar a capacidade, possibilitamos experiências de aprendizagem práticas que preparam as mentes jovens para um mundo em que a criatividade humana e a precisão das máquinas trabalham em harmonia. A nossa convicção fundamental é que todos os estudantes devem ter a oportunidade de experimentar os blocos de construção da automação moderna - da robótica e dos sistemas CNC aos instrumentos científicos. Através das nossas parcerias educativas e da nossa abordagem de código aberto, estamos a formar uma geração que não se limita a usar a tecnologia, mas que verdadeiramente a compreende e a molda. Juntos, estamos a construir a base para os inovadores que irão definir um futuro extraordinário.

Código Aberto

Acreditamos em tornar o mundo melhor através da tecnologia. O software, o firmware e os ficheiros de desenho das PCB estão disponíveis aqui:



<https://github.com/tomrodinger/servomotor>



Versão da Ficha Técnica: 1.9 Data de Publicação: Jun. 29, 2026

(c) 2026 Todas as especificações estão sujeitas a alteração sem aviso prévio.